

3.6 — Integrate Claude Code into CI/CD Pipelines

Concept

Claude Code is primarily an interactive tool, but its non-interactive mode makes it usable inside automated pipelines. CI/CD integration relies on four key capabilities: the `-p` flag for non-interactive prompt execution, `-output-format json` for machine-readable output, `--json-schema` for enforcing structured response shapes, and stateless session isolation that prevents context leakage between pipeline runs.

The `-p` Flag (Non-Interactive Mode)

The `-p` flag passes a prompt directly on the command line and exits after receiving the response. There is no REPL, no waiting for user input, and the process exits with a non-zero code on failure — making it composable with standard shell scripting and pipeline tooling.

```
claude -p "Review the following diff for security issues and list any findings"
```

Combined with stdin, you can pipe context into the prompt:

```
git diff HEAD~1 | claude -p "Summarize this diff as a pull request description. Be concise."
```

`--output-format json`

By default, Claude Code writes conversational prose to stdout. In CI, you need machine-readable output that scripts can parse. The `--output-format json` flag wraps the response in a JSON envelope:

```
git diff HEAD~1 | claude -p "List changed files and one-line summaries" --output-format json
```

Output shape:

```
{
  "role": "assistant",
  "content": "...",
  "stop_reason": "end_turn"
}
```

Parse the content field with `jq` to extract values for downstream steps:

```
SUMMARY=$(git diff HEAD~1 | claude -p "One-line summary of changes" --output-format json | jq -
```

`--json-schema`

When you need the response to conform to a specific data shape — not just be valid JSON — pass a JSON Schema file with `--json-schema`. Claude will return a response that validates against the schema, enabling safe deserialization into typed structures.

```
claude -p "Analyze this code for issues" \  
  --output-format json \  
  --json-schema ./schemas/code-review.json \  
< src/main.py
```

Example schema (`code-review.json`):

```
{  
  "type": "object",  
  "required": ["severity", "issues"],  
  "properties": {  
    "severity": { "type": "string", "enum": ["none", "low", "medium", "high", "critical"] },  
    "issues": {  
      "type": "array",  
      "items": {  
        "type": "object",  
        "required": ["file", "line", "description"],  
        "properties": {  
          "file": { "type": "string" },  
          "line": { "type": "integer" },  
          "description": { "type": "string" }  
        }  
      }  
    }  
  }  
}
```

Session Context Isolation

Interactive Claude Code sessions maintain conversation history across turns. In CI, each `claude -p` invocation is completely stateless — there is no persistent session, no memory of previous runs, and no shared context between jobs. This is the correct default for pipelines: it prevents stale context from one PR contaminating another, and makes runs reproducible.

If a single pipeline step requires multi-turn context, chain prompts explicitly by passing prior output as input in the same command, rather than relying on session resumption.

Code Example

Environment Setup

```
export ANTHROPIC_API_KEY="sk-ant-..." # set in CI secrets, never hardcoded
```

GitHub Actions Workflow

```
name: AI Code Review

on:
  pull_request:
    types: [opened, synchronize]

jobs:
  claude-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Install Claude Code
        run: npm install -g @anthropic-ai/claude-code

      - name: Generate PR description
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          git diff origin/main...HEAD | claude -p \
            "Write a concise GitHub pull request description with a summary and bullet-point list of changes." \
            --output-format json | jq -r '.content' > pr_description.txt
          cat pr_description.txt

      - name: Run automated code review
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          REVIEW=$(git diff origin/main...HEAD | claude -p \
            "Review this diff for bugs, security issues, and style problems. Return JSON." \
            --output-format json \
            --json-schema .github/schemas/review.json)
          SEVERITY=$(echo "$REVIEW" | jq -r '.severity')
          echo "Review severity: $SEVERITY"
          if [ "$SEVERITY" = "critical" ]; then
```

```

        echo "Critical issues found – failing build"
        exit 1
    fi

- name: Generate missing tests
  env:
    ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
  run: |
    git diff origin/main...HEAD -- '*.ts' | claude -p \
      "For each changed function, write a Jest unit test. Output only the test code." \
      > generated_tests.ts

```

Common Mistakes

Hardcoding the API key. Never write `ANTHROPIC_API_KEY=sk-ant-...` in YAML or scripts. Store it as a CI secret and inject it as an environment variable.

Expecting session continuity. Each `-p` invocation is independent. Do not assume that a second `claude -p` call in the same job has access to the context from the first. Pass all needed context explicitly in the prompt or via stdin.

Parsing prose output as JSON. Without `--output-format json`, Claude writes readable text. Piping that text to `jq` will fail. Always pair `--output-format json` with any downstream JSON parsing.

Omitting `--json-schema` for typed pipelines. `--output-format json` wraps the response in an envelope but does not constrain the content shape. For safe deserialization, always provide `--json-schema` when downstream code expects specific fields.

Ignoring exit codes. Claude Code exits non-zero on API errors, rate limits, or schema validation failures. CI steps must check the exit code or use `set -e` to catch failures rather than silently continuing with empty output.

Sending unbounded diffs. Very large diffs can exceed context limits or produce slow, expensive responses. Limit scope with `git diff -- '*.py'` or break reviews into per-file steps for large PRs.

References

- Claude Code CLI reference: `claude --help`
- Anthropic docs: [Claude Code in CI/CD](#)
- JSON Schema specification: json-schema.org
- Exam objectives: Section 3 — Automation and Pipeline Integration